(since thats what `max_points` was when the assignment is made) (I use the key `#Max` since `#` is sorted ahead of any alphabetic characters). Next `print_menu` is defined. Next the `print_all_grades` function is defined in the lines:

```
def print_all_grades():
    print('\t',end=" ")
    for i in range(len(assignments)):
        print(assignments[i], '\t',end=" ")
    print()
    keys = list(students.keys())
    keys.sort()
    for x in keys:
        print(x, '\t',end=' ')
        grades = students[x]
        print_grades(grades)
```

Notice how first the keys are gotten out of the `students` dictionary with the `keys` function in the line `keys = list(students.keys())`. `keys` is an iterable, and it is converted to list so all the functions for lists can be used on it. Next the keys are sorted in the line `keys.sort()`. `for` is used to go through all the keys. The grades are stored as a list inside the dictionary so the assignment `grades = students[x]` gives `grades` the list that is stored at the key `x`. The function `print_grades` just prints a list and is defined a few lines later.

The later lines of the program implement the various options of the menu. The line `students[name] = [0] * len(max_points)` adds a student to the key of their name. The notation `[0] * len(max_points)` just creates a list of 0's that is the same length as the `max_points` list.

The remove student entry just deletes a student similar to the telephone book example. The record grades choice is a little more complex. The grades are retrieved in the line `grades = students[name]` gets a reference to the grades of the student `name`. A grade is then recorded in the line `grades[which] = grade`. You may notice that `grades` is never put back into the students dictionary (as in no `students[name] = grades`). The reason for the missing statement is that `grades` is actually another name for `students[name]` and so changing `grades` changes `student[name]`.

Dictionaries provide an easy way to link keys to values. This can be used to easily keep track of data that is attached to various keys.

14 Using Modules

Here's this chapter's typing exercise (name it cal.py (`import` actually looks for a file named calendar.py and reads it in. If the file is named calendar.py and it sees a "import calendar" it tries to read in itself which works poorly at best.)):

```
import calendar
year = int(input("Type in the year number: "))
calendar.prcal(year)
```

And here is part of the output I got:

```
Type in the year number: 2001

                                  2001

       January                   February                  March

Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
1  2  3  4  5  6  7                1  2  3  4                1  2  3  4
8  9 10 11 12 13 14       5  6  7  8  9 10 11       5  6  7  8  9 10 11
15 16 17 18 19 20 21      12 13 14 15 16 17 18      12 13 14 15 16 17 18
22 23 24 25 26 27 28      19 20 21 22 23 24 25      19 20 21 22 23 24 25
29 30 31                  26 27 28                  26 27 28 29 30 31
```

(I skipped some of the output, but I think you get the idea.) So what does the program do? The first line `import calendar` uses a new command `import`. The command `import` loads a module (in this case the `calendar` module). To see the commands available in the standard modules either look in the library reference for python (if you downloaded it) or go to `http://docs.python.org/3/library/`. If you look at the documentation for the calendar module, it lists a function called `prcal` that prints a calendar for a year. The line `calendar.prcal(year)` uses this function. In summary to use a module `import` it and then use `module_name.function` for functions in the module. Another way to write the program is:

```
from calendar import prcal

year = int(input("Type in the year number: "))
prcal(year)
```

This version imports a specific function from a module. Here is another program that uses the Python Library (name it something like clock.py) (press Ctrl and the 'c' key at the same time to terminate the program):

```
from time import time, ctime

prev_time = ""
while True:
```

```
    the_time = ctime(time())
    if prev_time != the_time:
        print("The time is:", ctime(time()))
        prev_time = the_time
```

With some output being:

```
The time is: Sun Aug 20 13:40:04 2000
The time is: Sun Aug 20 13:40:05 2000
The time is: Sun Aug 20 13:40:06 2000
The time is: Sun Aug 20 13:40:07 2000

Traceback (innermost last):
 File "clock.py", line 5, in ?
    the_time = ctime(time())

KeyboardInterrupt
```

The output is infinite of course so I cancelled it (or the output at least continues until Ctrl+C is pressed). The program just does an infinite loop (`True` is always true, so `while True:` goes forever) and each time checks to see if the time has changed and prints it if it has. Notice how multiple names after the import statement are used in the line `from time import time, ctime`.

The Python Library contains many useful functions. These functions give your programs more abilities and many of them can simplify programming in Python.

14.0.1 Exercises

Rewrite the `high_low.py` program from section Decisions[1] to use an random integer between 0 and 99 instead of the hard-coded 7. Use the Python documentation to find an appropriate module and function to do this.

Solution

Rewrite `the high_low.py` program from section Decisions[2] to use an random integer between 0 and 99 instead of the hard-coded 7. Use the Python documentation to find an appropriate module and function to do this.

```
from random import randint
number = randint(0, 99)
guess = -1
while guess != number:
    guess = int(input ("Guess a number: "))
    if guess > number:
        print("Too high")
    elif guess < number:
            print("Too low")
```

1 Chapter 6.0.2 on page 32
2 Chapter 6.0.2 on page 32